Great. We'll start with Topic 1: Variables & Data Types. I'll follow the same structure for every topic.

JavaScript Notes

Chapter 1: Variables & Data Types

1. Definition

A variable is a named container used to store data in memory. The value stored in a variable can be changed (depending on how it is declared).

A data type defines the type of value that a variable can hold, such as numbers, text, or boolean values.

---

2. Why Do We Use Variables?

Variables help us:

Store data

Reuse values

Perform calculations

Make programs dynamic

Manage application state

Example:

```
let username = "John";
let age = 22;

console.log(username);
console.log(age);
```

---

3. Types of Variable Declarations

JavaScript provides three ways to declare variables.

A. var

Definition

var is the oldest way to declare variables. It is function-scoped and can be redeclared.

Syntax

```
var variableName = value;
```

Example

```
var city = "Delhi";
var city = "Mumbai";
```

```
console.log(city);
```

Output

```
Mumbai
```

Features

Function scoped

Can be redeclared

Can be reassigned

Hoisted with undefined

Avoid using in modern JavaScript

B. let

Definition

let is introduced in ES6. It is block-scoped and can be reassigned but cannot be redeclared in the same scope.

Syntax

```
let variableName = value;
```

Example

```
let age = 20;
age = 25;

console.log(age);
```

Output

25

Features

Block scoped

Cannot be redeclared

Can be reassigned

Preferred for changing values

C. const

Definition

const creates a constant variable. It cannot be reassigned after initialization.

Syntax

const variableName = value;

Example

const PI = 3.14159;

console.log(PI);

Output

3.14159

Features

Block scoped

Cannot be redeclared

Cannot be reassigned

Must be initialized during declaration

---

4. Difference Between var, let, and const

Feature var let const

Scope Function Block Block

Redeclare ✅ Yes ❌ No ❌ No

Reassign ✅ Yes ✅ Yes ❌ No
Hoisted ✅ Yes ✅ Yes (TDZ) ✅ Yes (TDZ)
Modern Use ❌ Avoid ✅ Yes ✅ Yes

---

5. JavaScript Data Types

JavaScript has 8 data types.

Primitive Data Types

1. String

Stores text.

let name = "John";

Example

console.log(typeof name);

Output

string

---

2. Number

Stores integers and decimal values.

```
let age = 25;
let price = 99.99;
```

Example

```
console.log(typeof age);
```

Output

```
number
```

---

3. Boolean

Stores true or false.

```
let isLoggedIn = true;
```

Example

```
console.log(typeof isLoggedIn);
```

Output

```
boolean
```

---

4. Undefined

A variable declared but not assigned a value.

let x;

Example

console.log(x);

Output

undefined

---

5. Null

Represents an intentional empty value.

let data = null;

Example

console.log(data);

Output

null

---

6. Symbol (ES6)

Used to create unique identifiers.

```
let id = Symbol("id");
```

---

7. BigInt

Stores very large integers.

```
let number = 12345678901234567890n;
```

Non-Primitive Data Type

8. Object

Stores collections of data.

```
let student = {
    name: "Rahul",
    age: 21
};
```

---

6. typeof Operator

Definition

The typeof operator returns the data type of a variable.

Syntax

typeof variableName;

Example

```
console.log(typeof "Hello");
console.log(typeof 10);
console.log(typeof true);
```

Output

```
string
number
boolean
```

---

7. Naming Rules for Variables

✅ Allowed

```
let userName;
let _price;
let $amount;
let student1;
```

❌ Not Allowed

```
let 1name;
let user-name;
let let;
```

Rules:

Must start with a letter, _, or $

Cannot start with a number

Case-sensitive

No spaces allowed

Reserved keywords cannot be used

---

8. Best Practices

Use const by default.

Use let only when the value changes.

Avoid using var.

Use meaningful variable names.

Follow camelCase naming convention.

Example

```
const firstName = "John";
let totalMarks = 450;
```

---

9. Common Mistakes

Forgetting Initialization

```
const age;
```

❌ Error

Redeclaring let

```
let x = 10;
let x = 20;
```

❌ Error

Reassigning const

```
const PI = 3.14;

PI = 3.1415;
```

❌ Error

---

10. Real-World Implementation

Variables are used in almost every JavaScript application.

Examples:

Login systems

Registration forms

Shopping carts

User profiles

Banking applications

Games

API responses

Dashboard statistics

---

11. Interview Questions

Q1. What is a variable?

A variable is a named container used to store data in memory.

Q2. Difference between let and const?

let can be reassigned.

const cannot be reassigned after initialization.

Q3. Why should we avoid var?

Because it is function-scoped, can be redeclared, and may lead to unexpected bugs due to hoisting.

Q4. What are primitive data types?

String, Number, Boolean, Undefined, Null, Symbol, and BigInt.

Q5. What does typeof do?

It returns the type of a variable or value.

---

12. Key Points (Revision)

A variable stores data.

JavaScript has 8 data types.

let is block-scoped and reassignable.

const is block-scoped and cannot be reassigned.

Avoid using var in modern JavaScript.

Use typeof to check a value's data type.

Prefer meaningful variable names and camelCase.

Next Chapter: Operators in JavaScript (Arithmetic, Assignment, Comparison, Logical, Bitwise, Ternary, Nullish Coalescing, Optional Chaining, and more).

JavaScript Notes

Chapter 2: Operators in JavaScript

---

1. Definition

An operator is a special symbol that performs an operation on one or more operands (values or variables) and returns a result.

Example:

```
let sum = 10 + 20;
```

Here:

+ → Operator

10 and 20 → Operands

30 → Result

---

2. Why Do We Use Operators?

Operators help us to:

Perform calculations

Compare values

Assign values

Make decisions

Work with logical conditions

Manipulate data

---

3. Types of Operators

JavaScript provides the following operators:

1. Arithmetic Operators

2. Assignment Operators

3. Comparison Operators

4. Logical Operators

5. Increment & Decrement Operators

6. String Operators

7. Ternary Operator

8. Nullish Coalescing Operator

9. Optional Chaining Operator

10. Bitwise Operators

11. Type Operators

---

4. Arithmetic Operators

Definition

Arithmetic operators perform mathematical calculations.

Operator Meaning Example

+ Addition 5 + 2
- Subtraction 5 - 2

*	Multiplication	5 * 2
/	Division	10 / 2
%	Modulus (Remainder)	10 % 3
**	Exponentiation	2 ** 3

Syntax

```
operand1 operator operand2;
```

Example

```
let a = 20;
let b = 5;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
console.log(a ** b);
```

Output

```
25
15
100
```

4

0

3200000

Features

Performs mathematical operations

Returns a new value

Works with numbers

---

5. Assignment Operators

Definition

Assignment operators assign values to variables.

Operator	Example	Meaning
=	x = 10	Assign
+=	x += 5	Add and Assign
-=	x -= 5	Subtract and Assign
*=	x *= 5	Multiply and Assign
/=	x /= 5	Divide and Assign
%=	x %= 5	Modulus and Assign

**= x **= 2 Power and Assign

Example

```
let x = 10;

x += 5;
x -= 2;
x *= 3;

console.log(x);
```

Output

```
39
```

Features

Reduces code length

Improves readability

Faster to write

---

6. Comparison Operators

Definition

Comparison operators compare two values and always return true or false.

Operator Meaning

== Equal Value
=== Equal Value & Type
!= Not Equal
!== Not Equal Value & Type
> Greater Than
< Less Than
>= Greater Than or Equal
<= Less Than or Equal

Example

```
console.log(10 == "10");
console.log(10 === "10");
console.log(10 != 5);
console.log(20 > 10);
```

Output

true

false

true

true

Key Point

Always prefer === over == because it checks both value and data type.

---

7. Logical Operators

Definition

Logical operators combine multiple conditions.

Operator	Meaning
&&	AND
||	OR
!	NOT

Example

```
let age = 20;
```

```
let citizen = true;

console.log(age >= 18 && citizen);
console.log(age < 18 || citizen);
console.log(!citizen);
```

Output

```
true
true
false
```

Features

Used in conditions

Returns Boolean value

Commonly used with if statements

---

8. Increment & Decrement Operators

Increment (++)

Increases value by 1.

```
let x = 5;
x++;

console.log(x);
```

Output

```
6
```

Decrement (--)

Decreases value by 1.

```
let x = 5;
x--;

console.log(x);
```

Output

```
4
```

Prefix vs Postfix

```
let a = 5;

console.log(++a);
```

Output

6

```
let b = 5;

console.log(b++);
console.log(b);
```

Output

```
5
6
```

---

9. String Operator

The + operator joins strings.

Example

```
let first = "Hello";
let second = "World";

console.log(first + " " + second);
```

Output

Hello World

---

10. Ternary Operator

Definition

A shorthand way of writing an if...else statement.

Syntax

condition ? value1 : value2;

Example

let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);

Output

Adult

Features

Short code

Easy to read

Best for simple conditions

---

11. Nullish Coalescing Operator (??)

Definition

Returns the right value only if the left value is null or undefined.

Example

```
let name = null;

console.log(name ?? "Guest");
```

Output

Guest

---

12. Optional Chaining (?.)

Definition

Safely accesses object properties without throwing an error if a property is missing.

Example

```
const user = {};

console.log(user.address?.city);
```

Output

```
undefined
```

---

13. Bitwise Operators

Bitwise operators work on binary numbers.

Operator	Meaning
&	AND
|	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

Example

```
console.log(5 & 1);
```

Output

1

---

14. Type Operators

typeof

Returns the data type.

console.log(typeof 100);

Output

number

---

instanceof

Checks whether an object belongs to a specific class.

let arr = [];

console.log(arr instanceof Array);

Output

true

---

15. Operator Precedence

JavaScript follows precedence rules when evaluating expressions.

Example

```
console.log(5 + 3 * 2);
```

Output

11

Explanation:

Multiplication (*) has higher precedence than addition (+).

---

16. Best Practices

Use === instead of ==.

Use ?? for default values instead of || when 0, false, or "" are valid values.

Use ternary operators only for simple conditions.

Write expressions clearly to improve readability.

---

17. Common Mistakes

Using ==

```
console.log(5 == "5");
```

Output

true

Better:

```
console.log(5 === "5");
```

Output

false

Confusing = with ==

Incorrect:

```
if (x = 10) {
    console.log("Wrong");
}
```

Correct:

```
if (x === 10) {
    console.log("Correct");
}
```

---

18. Real-World Implementation

Operators are used in:

Login validation

Age verification

Form validation

Shopping cart calculations

Salary calculations

Banking systems

Search filters

E-commerce discounts

API response handling

---

19. Interview Questions

Q1. What is an operator?

An operator is a symbol that performs an operation on one or more operands.

Q2. Difference between == and ===?

== compares only values after type coercion.

=== compares both value and data type without type coercion.

Q3. What does ?? do?

It returns the right-hand value only if the left-hand value is null or undefined.

Q4. What is the difference between && and ||?

&& returns true only if all conditions are true.

|| returns true if at least one condition is true.

Q5. What is the ternary operator?

A shorthand syntax for writing simple if...else conditions.

---

20. Key Points (Revision)

Operators perform actions on values.

Arithmetic operators are used for calculations.

Assignment operators update variable values.

Comparison operators return true or false.

Logical operators combine conditions.

Prefer === over ==.

Use ?? for safe default values.

Use ?. to safely access nested object properties.

Ternary operators simplify simple conditional logic.

Next Chapter: Strings in JavaScript (String methods, template literals, escape characters, searching, slicing, replacing, and more).

JavaScript Notes

Chapter 3: Strings in JavaScript

---

1. Definition

A String is a primitive data type used to store textual data (characters, words, or sentences). Strings are enclosed in single quotes ('), double quotes ("), or backticks ( ).

Examples:

```
let firstName = "John";
let lastName = 'Doe';
let message = `Welcome`;
```

---

2. Why Do We Use Strings?

Strings are used to:

Store names

Display messages

Handle user input

Format data

Build URLs

Process text

Create templates

---

3. Creating Strings

Using Double Quotes

Syntax

let name = "John";

Example

let city = "Delhi";
console.log(city);

Output

Delhi

Using Single Quotes

Syntax

```
let name = 'John';
```

Example

```
let language = 'JavaScript';
console.log(language);
```

Output

```
JavaScript
```

Using Backticks (Template Literals)

Definition

Backticks allow multiline strings and string interpolation.

Syntax

```
let text = `Hello`;
```

Example

```
let name = "Rahul";
console.log(`Welcome ${name}`);
```

Output

Welcome Rahul

---

4. String Properties

length

Definition

Returns the total number of characters in a string.

Syntax

string.length

Example

let text = "JavaScript";

console.log(text.length);

Output

10

---

5. Accessing Characters

Using Index

Syntax

```
string[index]
```

Example

```
let text = "Hello";

console.log(text[0]);
console.log(text[4]);
```

Output

```
H
o
```

---

charAt()

Definition

Returns the character at a specified index.

Syntax

```
string.charAt(index)
```

Example

```
let text = "JavaScript";
```

```
console.log(text.charAt(4));
```

Output

```
S
```

---

6. Changing Case

toUpperCase()

Converts the string to uppercase.

```
let text = "hello";

console.log(text.toUpperCase());
```

Output

```
HELLO
```

---

toLowerCase()

Converts the string to lowercase.

```
let text = "HELLO";
```

```
console.log(text.toLowerCase());
```

Output

```
hello
```

---

7. Removing Spaces

trim()

Removes spaces from both ends.

```
let text = "   JavaScript   ";

console.log(text.trim());
```

Output

```
JavaScript
```

---

trimStart()

Removes spaces from the beginning.

```
let text = "   Hello";

console.log(text.trimStart());
```

---

trimEnd()

Removes spaces from the end.

```
let text = "Hello   ";

console.log(text.trimEnd());
```

---

8. Searching Strings

includes()

Checks if a string contains a value.

```
let text = "JavaScript";

console.log(text.includes("Script"));
```

Output

true

---

startsWith()

Checks whether a string starts with specific characters.

```
let text = "JavaScript";

console.log(text.startsWith("Java"));
```

Output

```
true
```

---

endsWith()

Checks whether a string ends with specific characters.

```
let text = "JavaScript";

console.log(text.endsWith("Script"));
```

Output

```
true
```

---

indexOf()

Returns the first matching index.

```
let text = "JavaScript";

console.log(text.indexOf("S"));
```

Output

```
4
```

---

lastIndexOf()

Returns the last matching index.

```
let text = "Hello Hello";

console.log(text.lastIndexOf("Hello"));
```

Output

```
6
```

---

9. Extracting Parts of Strings

slice()

Extracts part of a string.

Syntax

string.slice(start, end)

Example

```
let text = "JavaScript";

console.log(text.slice(0,4));
```

Output

Java

---

substring()

Returns characters between indexes.

```
let text = "JavaScript";

console.log(text.substring(4,10));
```

Output

Script

---

10. Replacing Text

replace()

Replaces the first occurrence.

```
let text = "I love Java";

console.log(text.replace("Java","JavaScript"));
```

Output

I love JavaScript

---

replaceAll()

Replaces every occurrence.

```
let text = "Cat Cat Cat";

console.log(text.replaceAll("Cat","Dog"));
```

Output

Dog Dog Dog

---

11. Splitting Strings

split()

Converts a string into an array.

```
let text = "HTML,CSS,JavaScript";

console.log(text.split(","));
```

Output

```
["HTML","CSS","JavaScript"]
```

---

12. Joining Strings

concat()

Joins two or more strings.

```
let first = "Hello";
let second = "World";

console.log(first.concat(" ", second));
```

Output

```
Hello World
```

---

13. Repeating Strings

repeat()

Repeats a string.

```
let text = "Hi ";

console.log(text.repeat(3));
```

Output

```
Hi Hi Hi
```

---

14. Template Literals

Definition

Template literals allow embedding variables and expressions inside strings using ${}.

Example

```
let name = "John";
let age = 22;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Output

```
My name is John and I am 22 years old.
```

Features

Supports multiline strings

Easy variable interpolation

Improves readability

---

15. Escape Characters

Escape	Meaning
\'	Single quote
\"	Double quote
\\	Backslash
\n	New line
\t	Tab

Example

```
console.log("Hello\nWorld");
```

Output

```
Hello
World
```

---

16. Best Practices

Use template literals instead of string concatenation.

Use trim() to clean user input.

Prefer includes() over indexOf() when checking for text.

Use meaningful variable names.

---

17. Common Mistakes

Forgetting Quotes

❌ Incorrect

let name = John;

✅ Correct

let name = "John";

Using + Instead of Template Literals

❌

```
let name = "Rahul";

console.log("Hello " + name);
```

✅

```
console.log(`Hello ${name}`);
```

---

18. Real-World Implementation

Strings are used in:

Login forms

Usernames

Email addresses

Password validation

Search functionality

Chat applications

Product names

URLs

API responses

Notifications

---

19. Interview Questions

Q1. What is a string?

A string is a primitive data type used to store textual data.

Q2. Difference between slice() and substring()?

slice() supports negative indexes.

substring() does not support negative indexes.

Q3. What does trim() do?

It removes whitespace from both the beginning and the end of a string.

Q4. What is a template literal?

A template literal is a string enclosed in backticks that supports multiline text and variable interpolation using ${}.

Q5. What is the difference between replace() and replaceAll()?

replace() replaces only the first occurrence.

replaceAll() replaces all matching occurrences.

---

20. Key Points (Revision)

A string stores textual data.

Strings are immutable (methods return new strings without changing the original).

Use .length to get the number of characters.

slice() extracts part of a string.

replaceAll() replaces every matching occurrence.

split() converts a string into an array.

Template literals (```) make string formatting easier.

Use trim() to remove unwanted spaces.

Next Chapter: Arrays in JavaScript

We'll cover:

Definition

Creating Arrays

Accessing Elements

Updating Elements

Array Properties

Nested Arrays

Multidimensional Arrays

Array Traversal

Real-world Examples

Best Practices

Common Mistakes

Interview Questions

JavaScript Notes

Chapter 4: Arrays in JavaScript

---

1. Definition

An Array is a special object in JavaScript used to store multiple values in a single variable. Each value in an array is called an element, and every element has an index starting from 0.

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
```

---

2. Why Do We Use Arrays?

Arrays help us to:

Store multiple values together.

Access elements using indexes.

Iterate over collections of data.

Perform searching, sorting, and filtering.

Manage lists efficiently.

---

3. Creating an Array

Method 1: Array Literal (Recommended)

Syntax

let arrayName = [value1, value2, value3];

Example

let fruits = ["Apple", "Banana", "Orange"];

console.log(fruits);

Output

["Apple", "Banana", "Orange"]

Method 2: Array Constructor

Syntax

let arrayName = new Array(value1, value2);

Example

```
let numbers = new Array(10, 20, 30);

console.log(numbers);
```

Output

```
[10, 20, 30]
```

---

4. Accessing Array Elements

Array elements are accessed using their index.

Syntax

```
arrayName[index];
```

Example

```
let colors = ["Red", "Blue", "Green"];

console.log(colors[0]);
console.log(colors[2]);
```

Output

```
Red
Green
```

---

5. Updating Array Elements

Example

```
let fruits = ["Apple", "Banana"];

fruits[1] = "Mango";

console.log(fruits);
```

Output

```
["Apple", "Mango"]
```

---

6. Array Property

length

Definition

Returns the total number of elements in an array.

Syntax

```
arrayName.length;
```

Example

```
let numbers = [10, 20, 30, 40];

console.log(numbers.length);
```

Output

```
4
```

---

7. Arrays Can Store Different Data Types

```
let data = [
    "John",
    25,
    true,
    null,
    { city: "Delhi" },
    [10, 20]
];

console.log(data);
```

---

8. Nested Arrays

A nested array is an array inside another array.

Example

```
let students = [
    ["John", 20],
    ["Rahul", 22],
    ["Amit", 19]
];

console.log(students[1][0]);
```

Output

Rahul

---

9. Multidimensional Arrays

```
let matrix = [
    [1,2,3],
    [4,5,6],
    [7,8,9]
];

console.log(matrix[2][1]);
```

Output

8

---

10. Traversing an Array

Using for Loop

```
let fruits = ["Apple","Banana","Mango"];

for(let i = 0; i < fruits.length; i++){
    console.log(fruits[i]);
}
```

Output

```
Apple
Banana
Mango
```

Using for...of Loop

```
let fruits = ["Apple","Banana","Mango"];

for(const fruit of fruits){
    console.log(fruit);
}
```

Using forEach()

```
let fruits = ["Apple","Banana","Mango"];
```

```
fruits.forEach(function(fruit){
    console.log(fruit);
});
```

---

11. Checking Array Type

Array.isArray()

Definition

Checks whether a value is an array.

Example

```
let arr = [1,2,3];

console.log(Array.isArray(arr));
```

Output

true

---

12. Difference Between Array and Object

Array	Object
Stores ordered data	Stores key-value pairs

Uses indexes Uses keys

Starts from index 0 Uses property names

Best for lists Best for structured data

---

13. Real-World Examples

Student Names

```
let students = [
    "Rahul",
    "Amit",
    "John"
];
```

Shopping Cart

```
let cart = [
    "Laptop",
    "Mouse",
    "Keyboard"
];
```

User IDs

```
let ids = [101,102,103,104];
```

Product List

```
let products = [
    "Phone",
    "Watch",
    "Shoes"
];
```

---

14. Features

Stores multiple values.

Ordered collection.

Dynamic size.

Supports different data types.

Zero-based indexing.

Built-in methods available.

Easy to iterate.

---

15. Best Practices

Use array literals ([]) instead of new Array().

Use meaningful variable names.

Use const if the array reference doesn't change.

Prefer for...of or forEach() for readability.

Keep arrays focused on related data.

---

16. Common Mistakes

Accessing an Invalid Index

```
let arr = [10,20,30];

console.log(arr[5]);
```

Output

```
undefined
```

Using Strings as Indexes

❌ Incorrect

```
let arr = [10,20];

console.log(arr["0"]);
```

Although it works due to type coercion, use numeric indexes for clarity.

Confusing Arrays with Objects

❌ Incorrect

```
let user = ["John",25];
```

If the data has named properties, use an object instead:

```
let user = {
    name: "John",
    age: 25
};
```

---

17. Real-World Implementation

Arrays are widely used in:

Shopping carts

Product listings

Student records

Employee management

Chat applications

To-do lists

API responses

Image galleries

Music playlists

Social media feeds

---

18. Interview Questions

Q1. What is an array?

An array is an ordered collection of values stored in a single variable.

Q2. Does array indexing start from 1?

No. Array indexing starts from 0.

Q3. Can an array store different data types?

Yes. JavaScript arrays can store strings, numbers, booleans, objects, arrays, functions, and more.

Q4. What does length return?

It returns the total number of elements in the array.

Q5. How do you check if a value is an array?

Array.isArray(value);

---

19. Key Points (Revision)

An array stores multiple values in a single variable.

Array indexes start at 0.

Use [] to create arrays.

Use .length to get the number of elements.

Arrays can contain mixed data types.

Use loops like for, for...of, or forEach() to traverse arrays.

Use Array.isArray() to verify if a value is an array.

Arrays are ideal for ordered collections of data.

Next Chapter: Array Methods in JavaScript

We'll cover all important methods in detail, including:

push()

pop()

shift()

unshift()

splice()

slice()

concat()

join()

includes()

indexOf()

find()

findIndex()

filter()

map()

reduce()

some()

every()

sort()

reverse()

flat()

flatMap()

fill()

copyWithin()

entries()

keys()

values()

Array.from()

Array.of()

JavaScript Notes

Chapter 5: Array Methods in JavaScript

---

1. Definition

Array Methods are built-in JavaScript functions that allow us to **add, remove, search, update, transform, sort, and manipulate** array elements efficiently.

---

2. Why Do We Use Array Methods?

Array methods help us to:

- Add or remove elements
- Search data
- Filter data
- Transform arrays
- Sort values
- Iterate through arrays
- Perform calculations

---

3. push()

Definition

Adds one or more elements **to the end** of an array.

Syntax

```
array.push(element1, element2);
```

Example

```
let fruits = ["Apple", "Banana"];

fruits.push("Mango");

console.log(fruits);
```

Output

```
["Apple", "Banana", "Mango"]
```

Features

- Adds elements at the end
- Modifies the original array
- Returns the new length

Real-world Implementation

- Add a product to a shopping cart
- Add a new message to a chat

---

4. pop()

Definition

Removes the **last element** from an array.

Syntax

```
array.pop();
```

Example

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.pop();

console.log(fruits);
```

Output

```
["Apple", "Banana"]
```

Features

- Removes last element
- Changes original array
- Returns removed element

---

5. shift()

Definition

Removes the **first element** from an array.

Syntax

```
array.shift();
```

Example

```
let numbers = [10,20,30];

numbers.shift();

console.log(numbers);
```

Output

```
[20,30]
```

Features

- Removes first element
- Re-indexes the array

---

6. unshift()

Definition

Adds one or more elements to the **beginning** of an array.

Syntax

```
array.unshift(element);
```

Example

```
let numbers = [20,30];

numbers.unshift(10);

console.log(numbers);
```

Output

```
[10,20,30]
```

Features

- Adds elements at the beginning
- Changes original array

---

7. concat()

Definition

Combines two or more arrays into a new array.

Syntax

```
array1.concat(array2);
```

Example

```
let a = [1,2];
let b = [3,4];

console.log(a.concat(b));
```

Output

```
[1,2,3,4]
```

Features

- Returns a new array
- Does not modify original arrays

---

8. slice()

Definition

Returns a selected portion of an array **without modifying** the original array.

Syntax

```
array.slice(start,end);
```

Example

```
let numbers = [10,20,30,40];

console.log(numbers.slice(1,3));
```

Output

```
[20,30]
```

Features

- Does not change original array
- Supports negative indexes

---

9. splice()

Definition

Adds, removes, or replaces elements in an array.

Syntax

```
array.splice(start, deleteCount, item1, item2);
```

Example (Remove)

```
let numbers = [10,20,30,40];

numbers.splice(1,2);

console.log(numbers);
```

Output

```
[10,40]
```

Example (Insert)

```
let fruits = ["Apple","Mango"];

fruits.splice(1,0,"Banana");

console.log(fruits);
```

Output

```
["Apple","Banana","Mango"]
```

Features

- Can add
- Can delete
- Can replace
- Changes original array

---

10. includes()

Definition

Checks whether an array contains a specific value.

Syntax

```
array.includes(value);
```

Example

```
let fruits = ["Apple","Banana"];

console.log(fruits.includes("Banana"));
```

Output

```
true
```

---

11. indexOf()

Definition

Returns the first index of an element.

Syntax

```
array.indexOf(value);
```

Example

```
let fruits = ["Apple","Banana","Mango"];

console.log(fruits.indexOf("Mango"));
```

Output

```
2
```

---

12. lastIndexOf()

Returns the last matching index.

```
let arr = [10,20,30,20];

console.log(arr.lastIndexOf(20));
```

Output

```
3
```

---

13. join()

Definition

Converts an array into a string.

Syntax

```
array.join(separator);
```

Example

```
let fruits = ["Apple","Banana","Mango"];

console.log(fruits.join("-"));
```

Output

```
Apple-Banana-Mango
```

---

14. reverse()

Definition

Reverses the order of elements.

Example

```
let numbers = [1,2,3];

numbers.reverse();

console.log(numbers);
```

Output

```
[3,2,1]
```

---

15. sort()

Definition

Sorts array elements.

Example

```
let numbers = [5,2,8,1];

numbers.sort((a,b)=>a-b);

console.log(numbers);
```

Output

```
[1,2,5,8]
```

Key Point

Always use a compare function when sorting numbers.

---

16. forEach()

Definition

Executes a function for each array element.

Syntax

```
array.forEach(callback);
```

Example

```
let numbers = [10,20,30];

numbers.forEach(function(num){
    console.log(num);
});
```

---

17. map()

Definition

Creates a **new array** by applying a function to every element.

Syntax

```
array.map(callback);
```

Example

```
let numbers = [1,2,3];

let square = numbers.map(num => num*num);

console.log(square);
```

Output

```
[1,4,9]
```

Features

- Returns a new array
- Does not modify original array

---

18. filter()

Definition

Returns a new array containing only elements that satisfy a condition.

Example

```
let numbers = [10,15,20,25];

let result = numbers.filter(num => num > 15);

console.log(result);
```

Output

```
[20,25]
```

---

19. find()

Definition

Returns the **first matching element**.

Example

```
let numbers = [5,10,15,20];

console.log(numbers.find(num => num > 10));
```

Output

```
15
```

---

20. findIndex()

Definition

Returns the index of the first matching element.

```
let numbers = [5,10,15,20];

console.log(numbers.findIndex(num => num > 10));
```

Output

```
2
```

---

21. some()

Definition

Returns `true` if **at least one** element satisfies the condition.

```
let numbers = [10,20,30];

console.log(numbers.some(num => num > 25));
```

Output

```
true
```

---

22. every()

Definition

Returns `true` only if **all elements** satisfy the condition.

```
let numbers = [10,20,30];

console.log(numbers.every(num => num > 5));
```

Output

```
true
```

---

23. reduce()

Definition

Reduces an array to a single value.

Syntax

```
array.reduce(callback, initialValue);
```

Example

```
let numbers = [10,20,30];
```

```
let total = numbers.reduce((sum,num)=>sum+num,0);

console.log(total);
```

Output

```
60
```

Real-world Implementation

- Shopping cart total
- Salary calculation
- Marks total
- Invoice generation

---

24. flat()

Definition

Flattens nested arrays.

```
let arr = [1,[2,[3]]];

console.log(arr.flat(2));
```

Output

```
[1,2,3]
```

---

25. flatMap()

Definition

Applies `map()` and then flattens the result by one level.

```
let numbers = [1,2,3];

let result = numbers.flatMap(num => [num, num * 2]);

console.log(result);
```

Output

```
[1,2,2,4,3,6]
```

---

26. fill()

Definition

Fills an array with a static value.

```
let arr = [1,2,3];

arr.fill(0);

console.log(arr);
```

Output

```
[0,0,0]
```

---

27. Array.from()

Definition

Creates an array from an iterable or array-like object.

```
console.log(Array.from("Hello"));
```

Output

```
["H","e","l","l","o"]
```

---

28. Array.of()

Definition

Creates an array from arguments.

```
let arr = Array.of(1,2,3);

console.log(arr);
```

Output

```
[1,2,3]
```

---

29. Best Practices

- Use `map()` to transform data.

- Use `filter()` to remove unwanted items.
- Use `find()` when you need the first matching element.
- Use `reduce()` for totals and aggregations.
- Prefer `slice()` when you don't want to modify the original array.
- Be careful with methods like `splice()`, `sort()`, and `reverse()` because they modify the original array.

---

30. Common Mistakes

❌ Using `map()` without storing its returned array.

❌ Using `filter()` when you only need one element (`find()` is better).

❌ Sorting numbers without a compare function.

```
[10, 2, 30].sort(); // Incorrect result: [10, 2, 30]
```

✅ Correct:

```
[10, 2, 30].sort((a, b) => a - b);
```

---

31. Interview Questions

Q1. Difference between `map()` and `forEach()`?

- `map()` returns a new array.
- `forEach()` returns `undefined`.

Q2. Difference between `slice()` and `splice()`?

- `slice()` does not modify the original array.
- `splice()` modifies the original array.

Q3. Difference between `find()` and `filter()`?

- `find()` returns the first matching element.
- `filter()` returns all matching elements in a new array.

Q4. Which method is used to calculate the total of array values?

`reduce()`

Q5. Which methods modify the original array?

Examples: `push()`, `pop()`, `shift()`, `unshift()`, `splice()`, `sort()`, `reverse()`, `fill()`.

---

32. Key Points (Revision)

- `push()` → Add to end.
- `pop()` → Remove from end.
- `shift()` → Remove from beginning.
- `unshift()` → Add to beginning.
- `slice()` → Copy part of an array (non-mutating).
- `splice()` → Add/remove/replace elements (mutating).
- `map()` → Transform elements into a new array.
- `filter()` → Select matching elements.
- `find()` → First matching element.
- `reduce()` → Reduce array to a single value.
- `sort()` → Sort elements (use a compare function for numbers).
- `Array.from()` → Create an array from iterable data.
- `Array.of()` → Create an array from arguments.

Next Chapter: Objects in JavaScript, where we'll cover object creation, properties, methods, nested objects, `this` keyword, object destructuring, and all major object methods (`Object.keys()`, `Object.values()`, `Object.entries()`, `Object.assign()`, `freeze()`, `seal()`, etc.).